

Git 学习笔记

2026 年 5 月 20 日

目录

1 引言——为什么要用 Git?	4
1.1 什么是版本控制?	4
1.2 集中式 vs 分布式	4
1.3 Git 的诞生	4
2 基本概念——理解 Git 的三个区域	5
2.1 工作区、暂存区与仓库	5
2.1.1 完整工作流程	5
2.2 Git 对象模型	5
2.2.1 四种核心对象	6
2.2.2 对象之间的关系	6
3 基本操作——从零开始用 Git	7
3.1 初始化与配置	7
3.1.1 第一次使用 Git 的必做配置	7
3.2 初始化仓库	7
3.3 文件的生命周期	8
3.4 查看状态——git status	8
3.5 添加文件——git add	9
3.6 提交——git commit	9
3.7 完整的新项目流程示例	10
3.8 查看历史——git log	10
3.9 比较差异——git diff	11
3.10 删除与移动	12
3.11 撤销操作	13
3.11.1 场景一：文件改了但还没 add（工作区级撤销）	13
3.11.2 场景二：已经 add 了但还没 commit（暂存区级撤销）	13
3.11.3 场景三：已经 commit 了但还没 push（仓库级撤销）	13
3.11.4 场景四：已经 push 了（安全的方式）	14

3.11.5 撤销操作选择速查表	14
4 分支管理——Git 最强大的特性	14
4.1 什么是分支?	14
4.2 为什么 Git 的分支那么轻量?	14
4.3 分支操作	15
4.4 合并——git merge	16
4.4.1 Fast-Forward 合并（直线合并）	16
4.4.2 Three-Way 合并（分叉合并）	16
4.5 合并冲突	17
4.5.1 制造冲突的示例	17
4.5.2 冲突标记	17
4.5.3 解决冲突的完整步骤	18
4.6 Rebase（变基）	18
4.6.1 Rebase vs Merge	18
4.6.2 交互式 Rebase ——改写历史的瑞士军刀	19
4.6.3 黄金法则——永远记住!	20
4.7 Cherry-Pick ——精准挑选提交	20
5 远程仓库——和世界协作	21
5.1 远程仓库的概念	21
5.2 克隆已有仓库	21
5.3 管理远程仓库	21
5.4 推送——git push	22
5.5 拉取——git pull 与 git fetch	22
5.6 标准协作流程	23
5.7 SSH 与 HTTPS 的区别	24
6 标签管理——标记重要版本	25
6.1 轻量标签 vs 附注标签	25
7 储藏（Stash）——临时保存工作现场	26
7.1 什么时候需要 Stash?	26

7.2	Stash 的更多用法	27
8	.gitignore ——告诉 Git 忽略什么	27
8.1	为什么要忽略文件?	27
8.2	语法规则	28
8.3	常见语言/项目的.gitignore 模板	29
9	实用技巧与最佳实践	30
9.1	别名 (Alias) ——少打字	30
9.2	查看日志的美化方式	30
9.3	常见场景速查	31
9.4	救命的 reflog	31
9.5	二分查找——git bisect	32
9.6	子模块 (Submodule)	33
9.7	提交信息规范——Conventional Commits	33
9.8	Git 钩子 (Hooks)	34
9.9	工作流模型	35
9.9.1	Git Flow ——适合有固定发布周期的项目	35
9.9.2	GitHub Flow ——适合持续部署	35
9.9.3	GitLab Flow	36
10	内部原理速览——Git 是如何工作的?	36
10.1	.git 目录全解析	36
10.2	底层命令实验	37
10.3	理解 git add、git commit 的底层操作	38
11	常见问题 FAQ	38
12	总结	39

1 引言——为什么要用 Git?

1.1 什么是版本控制?

假设你在写毕业论文，你一定经历过这样的场景：

```
毕业论文_初稿.docx
毕业论文_修改1.docx
毕业论文_修改2.docx
毕业论文_最终版.docx
毕业论文_真的最终版.docx
毕业论文_打死也不改了.docx
```

这就是**没有版本控制**的混乱状态。版本控制系统（VCS）就是为了解决这个问题而生——它帮你记录每次修改的历史，让你随时可以回到任意版本，还能多人并行协作而不互相覆盖。

1.2 集中式 vs 分布式

特性	集中式（SVN、CVS）	分布式（Git）
仓库位置	只有一个中央服务器	每个人本地都有完整仓库
离线工作	不行，必须联网	可以，大部分操作本地完成
速度	慢（大部分操作需要网络）	快（本地操作）
安全性	中央服务器挂了就全丢了	每个人都是备份
分支代价	高（完整复制整个项目）	极低（只是一个指针）

1.3 Git 的诞生

2005 年，Linux 内核社区和当时的版本控制系统 BitKeeper 闹翻了。Linus Torvalds 花了两周时间，用 C 语言写出了 Git。设计目标是：

- 速度飞快
- 设计简单（数据结构精妙但简洁）
- 完全分布式
- 支持非线性开发（成千上万个分支）

如今，Git 已经是全世界最流行的版本控制系统，无论个人项目还是企业级开发，几乎都在用。

2 基本概念——理解 Git 的三个区域

2.1 工作区、暂存区与仓库

这是 Git 最核心的抽象，理解了这三个区域，你就理解了 Git 的一半。

- 工作区 (Working Directory)

- 就是你电脑上看到的文件目录。
- 你在这里编辑文件、写代码、改 bug。
- 这是你可以直接看到、操作的区域。

- 暂存区 (Staging Area / Index)

- 可以理解为一个购物车。
- 你改了一堆文件，但不想一次性全提交。
- 用 `git add` 把想要提交的文件先放进购物车。
- 暂存区让你可以精心挑选每次提交的内容。

- 本地仓库 (Repository / .git)

- 就是隐藏的 `.git/` 目录。
- 存着所有的版本历史、分支信息、标签等。
- 相当于 Git 的数据库。
- `git commit` 就是把暂存区的内容写入这个数据库。

2.1.1 完整工作流程

初始状态	<code>git add</code> 之后	<code>git commit</code> 之后
工作区：修改了 A B C 三个文件	暂存区：A B C	暂存区：(空)
	工作区：A B C	本地仓库：(记录了
一次新版本)		工作区：A B C

打个比方：你在做饭（工作区），切好的菜放进盘子里（暂存区 — 随时可以再加或拿走），准备好后一起下锅炒（提交）。每炒好一道菜就是一个版本。

2.2 Git 对象模型

Git 本质上是一个内容寻址的文件系统，它用哈希值 (SHA-1) 来定位所有内容。

2.2.1 四种核心对象

1. blob (Binary Large Object)

- 存储文件内容。
- 不包含文件名，只是内容的二进制快照。
- 两个文件即使名字不同，如果内容相同，指向同一个 blob。

2. tree (树对象)

- 存储目录结构。
- 记录文件名 → blob 的映射，以及子目录 → tree 的映射。
- 相当于文件系统中的目录（文件夹）。

3. commit (提交对象)

- 存储一次提交快照。
- 包含：指向一棵 tree、父提交的哈希、作者信息、时间戳、提交消息。
- 每个 commit 就是一个版本。

4. tag (标签对象)

- 为某个 commit 起一个固定的名字（如 v1.0.0）。
- 分支会移动，但标签不会。

2.2.2 对象之间的关系

```
commit (abc123)
|-- 指向 tree (def456)
|-- 父提交：上一个 commit
|-- 作者/时间/消息
|
|   tree (def456)
|   |-- "main.c"    --> blob (aaa111)    # 文件内容
|   |-- "util.h"    --> blob (bbb222)
|   |-- "src/"      --> tree (ccc333)    # 子目录
```

所有的对象都通过 40 位的 SHA-1 哈希值标识，例如 e86a3764c7a5b5e8a440b8e139e4c0e6c2e4d6。Git 保证：只要内容不同，哈希值就不同。所以哈希值既是地址，也是校验和。

3 基本操作——从零开始用 Git

3.1 初始化与配置

3.1.1 第一次使用 Git 的必做配置

```
1 # Git 需要知道你是谁，因为每次提交都会记录作者信息
2 git config --global user.name "Your Name"
3 git config --global user.email "email@example.com"
4
5 # 查看所有配置
6 git config --list
7
8 # 建议：设置默认分支名为 main（旧版 Git 默认是 master）
9 git config --global init.defaultBranch main
```

配置文件有三个层级：

- `--system`：系统级，影响所有用户（很少用）
- `--global`：用户级，影响你的所有仓库（最常用）
- `--local`：仓库级，只影响当前仓库（默认）

优先级：local > global > system。

3.2 初始化仓库

```
1 # 进入你的项目目录
2 cd my-project
3
4 # 把当前目录变成 Git 仓库（会创建一个 .git/ 隐藏目录）
5 git init
6
7 # 执行后，你就可以开始用 Git 管理这个目录的文件了
```

`git init` 只是在当前目录创建了一个 `.git/` 文件夹，你的文件本身没有任何变化，Git 从此开始监视这个目录。

3.3 文件的生命周期

每个文件在 Git 中有四种状态：

Untracked（未跟踪）

- |-- 文件刚创建，Git 还没有记录它
- |-- `git status` 会显示为红色
- |-- `git add` 后变成 Staged
- |

Tracked（已跟踪）下的三种子状态：

- |-- Unmodified（未修改）：和仓库里的内容一样
- |-- Modified（已修改）：改了但还没 `add`
- |-- Staged（已暂存）：`add` 了但还没 `commit`

状态转换图：

```
Untracked --git add--> Staged --git commit--> Unmodified
Modified  --git add--> Staged
Unmodified --修改文件--> Modified
```

3.4 查看状态——`git status`

这是最常用的命令，没有之一。任何时候不确定当前状态，先敲它。

```
1 # 假设你刚 init, 目录里只有一个 README.md
2 $ git status
3 On branch main
4
5 No commits yet
6
7 Untracked files:
8   (use "git add <file>..." to include in what will be committed)
9     README.md
10
11 nothing added to commit but untracked files present
12
13 # 简洁模式
14 $ git status -s
15 ?? README.md          # ?? 表示 Untracked
```

状态符号含义：

- ?? —未跟踪
- M —已修改
- A —新添加到暂存区
- D —已删除
- R —已重命名
- 左列 = 暂存区状态，右列 = 工作区状态

3.5 添加文件——git add

```

1 # 添加单个文件
2 git add index.html
3
4 # 添加所有变更（新增、修改、删除）
5 git add .
6
7 # 添加当前目录的所有 .py 文件
8 git add *.py
9
10 # 交互式逐块添加（可以只提交文件的一部分修改！）
11 git add -p
12 # 这个命令会逐个"块"询问你是否要暂存
13 # y = 暂存这一块，n = 跳过，s = 再拆细，e = 手动编辑

```

git add -p 非常实用。比如你改了某个文件，有些修改是关于修复 bug A，有些是添加功能 B，你可以用 git add -p 只暂存 A 的部分，先提交 A，再暂存 B 的部分，提交 B。做到一次提交只做一件事。

3.6 提交——git commit

```

1 # 最简单的提交
2 git commit -m "修复了登录页面的 bug"
3
4 # 提交前自动 add 所有已跟踪文件的修改（不包含新文件）
5 git commit -a -m "更新配置文件和文档"
6
7 # 修改最近一次提交（如果漏了文件或提交消息写错了）
8 git commit --amend

```

9 # 注意：会改变提交的哈希值，不要用于已推送的提交！

提交的最佳实践：

- 每次提交只做一件事，保持原子性。
- 提交消息要说明为什么修改，而不是改了什么。
- 坏例子：修改了 `login.py`
- 好例子：修复了密码输入过长时的截断 bug

3.7 完整的新项目流程示例

```
1 # 1. 初始化
2 git init
3 git config --global user.name "张三"
4 git config --global user.email "zhangsan@example.com"
5
6 # 2. 创建文件并提交
7 echo "# 我的项目" > README.md
8 git add README.md
9 git commit -m "docs: 初始化项目说明"
10
11 # 3. 写了一些代码后再提交
12 git add main.py utils.py
13 git commit -m "feat: 添加核心功能和工具模块"
14
15 # 4. 改了一堆文件后统一提交
16 git add .
17 git commit -m "fix: 修复了三处边界条件处理问题"
18
19 # 5. 查看历史
20 git log --oneline
```

3.8 查看历史——git log

```
1 # 基本查看
2 git log
3
```

```

4 # 最常用的精简模式
5 git log --oneline
6 # 输出:
7 # 3a4b5c6 修复了登录 bug
8 # 2b3c4d5 添加了用户注册功能
9 # 1a2b3c4 初始化项目
10
11 # 图形化查看所有分支
12 git log --graph --oneline --all
13 # 输出:
14 # * 3a4b5c6 (HEAD -> main) 修复了登录 bug
15 # | * 4b5c6d7 (feature) 添加了新页面
16 # |/
17 # * 2b3c4d5 添加了用户注册功能
18
19 # 查看每个提交的详细差异
20 git log -p
21
22 # 查看文件变更统计
23 git log --stat
24 # 输出: xxx.py | 10 +++++-----
25
26 # 按条件筛选
27 git log --author="张三"
28 git log --since="2024-01-01" --until="2024-12-31"
29 git log --grep="bug"
30 git log -3 # 只看最近 3 次提交
31
32 # 查看某次提交的详细内容
33 git show 3a4b5c6 # 用提交哈希的前几位即可

```

3.9 比较差异——git diff

git diff 用来查看文件的具体修改内容，是 code review 和检查错误的好帮手。

```

1 # 工作区 vs 暂存区（还没 add 的修改）
2 git diff
3
4 # 暂存区 vs 最近一次提交（准备要提交的内容）
5 git diff --staged # 或 git diff --cached

```

```
6
7 # 工作区 vs 最近一次提交（所有未提交的修改）
8 git diff HEAD
9
10 # 两个分支之间的差异
11 git diff main feature
12
13 # 两次提交之间的差异
14 git diff 3a4b5c6 2b3c4d5
15
16 # 只看某个文件的差异
17 git diff -- README.md
18
19 # 忽略空格变化的差异
20 git diff -w
```

输出解读:

```
diff --git a/index.html b/index.html
index e69de29..40559da 100644
--- a/index.html          # a 版本
+++ b/index.html          # b 版本
@@ -0,0 +1,3 @@           # 从第0行开始，加了3行
+<html>                   # "+" 表示新增的行
+<head><title>Hello</title></head>
+</html>
```

3.10 删除与移动

```
1 # 删除文件（同时在文件系统删除 + Git 中记录删除）
2 git rm app.log
3
4 # 仅从 Git 中移除跟踪，保留本地文件（常见于 .gitignore 问题修复）
5 git rm --cached config.yml
6
7 # 重命名或移动文件
8 git mv oldname.txt newname.txt
9 # 等价于：
10 # mv oldname.txt newname.txt
11 # git rm oldname.txt
```

```
12 # git add newname.txt
```

3.11 撤销操作

撤销是 Git 初学者最常疑问的地方，关键在于区分撤销了什么区域。

3.11.1 场景一：文件改了但还没 add（工作区级撤销）

```
1 # 用仓库里的版本覆盖工作区的修改（危险！修改会丢失）
2 git restore README.md
3 # 旧式写法（功能一样）：
4 git checkout -- README.md
```

3.11.2 场景二：已经 add 了但还没 commit（暂存区级撤销）

```
1 # 从暂存区撤出，但保留工作区的修改
2 git restore --staged README.md
3 # 旧式写法：
4 git reset HEAD README.md
```

3.11.3 场景三：已经 commit 了但还没 push（仓库级撤销）

```
1 # --soft：只移动 HEAD，工作区和暂存区都不动
2 # 场景：提交后发现忘了加某个文件
3 git reset --soft HEAD~1
4 # 现在文件还在暂存区，你只需要重新 commit
5
6 # --mixed（默认）：移动 HEAD，清空暂存区但保留工作区
7 # 场景：想把提交拆成多个更小的提交
8 git reset --mixed HEAD~1
9
10 # --hard：完全撤销，丢弃所有修改（非常危险！）
11 git reset --hard HEAD~1
12 # 场景：提交了错误的代码，想完全不要这次修改
```

HEAD 1 表示 HEAD 的上一个提交，HEAD 3 表示往上数 3 个。

3.11.4 场景四：已经 push 了（安全的方式）

```
1 # 不要用 reset! 因为 reset 会改写历史, push 会被拒绝
2 # 正确做法: 用 revert 创建一个"反提交"
3 git revert HEAD          # 撤销最近一次提交
4 git revert 3a4b5c6       # 撤销指定提交
5
6 # revert 会创建一个新的提交, 内容是"把指定提交的修改反向应用"
7 # 这是完全安全的, 因为历史没有被改写
```

3.11.5 撤销操作选择速查表

状态	推荐命令	安全性
改了还没 add	git restore <file>	危险, 修改丢失
add 了还没 commit	git restore --staged <file>	安全
commit 了 还没 push	git reset HEAD~1	安全 (-hard 除外)
已经 push 了	git revert <commit>	最安全

4 分支管理——Git 最强大的特性

4.1 什么是分支？

分支本质上就是一个指向某次提交的指针。

类比：想象你正在写一本书（main 分支），突然想尝试另一个写法，但不想影响原来的版本。于是你复印了一份（创建分支），在复印件上修改。如果新写法好，就合并回原稿；如果不好，扔掉复印件即可。

4.2 为什么 Git 的分支那么轻量？

- SVN 创建分支 = 复制整个项目目录到另一个位置（慢，占空间）
- Git 创建分支 = 创建一个 41 字节的文件，里面只记录一个哈希值（瞬间完成）
- 所以 Git 鼓励频繁创建和合并分支

HEAD 是一个特殊指针，它指向你当前在哪个分支上。当你切换分支时，HEAD 就指向另一个分支。

```

# 当前在 main 分支
HEAD -> main -> commit (abc123)

# 创建 feature 分支后
HEAD -> main -> commit (abc123)
      feature -> commit (abc123)    # 两个指针指向同一个提交

# 在 feature 上提交后
HEAD -> main -> commit (abc123)
      feature -> commit (def456)    # feature 前进了

```

4.3 分支操作

```

1 # 查看分支
2 git branch                # 本地分支, * 表示当前所在分支
3 git branch -r             # 远程分支 (如 origin/main)
4 git branch -a             # 全部
5
6 # 创建分支
7 git branch feature-login  # 创建分支 (基于当前所在位置)
8
9 # 切换分支
10 git switch feature-login  # 新版命令 (推荐)
11 git checkout feature-login # 传统命令
12
13 # 创建并切换 (一行搞定)
14 git switch -c feature-login
15 git checkout -b feature-login # 传统写法
16
17 # 重命名分支
18 git branch -m old-name new-name
19
20 # 删除分支
21 git branch -d feature-login # 只能删除已合并的
22 git branch -D feature-login # 强制删除 (不管是否合并)
23
24 # 查看每个分支的最后一次提交
25 git branch -v

```

```

26
27 # 查看已合并到当前分支的分支
28 git branch --merged
29
30 # 查看未合并的分支
31 git branch --no-merged

```

4.4 合并——git merge

合并是把两个分支的工作成果整合在一起。

4.4.1 Fast-Forward 合并（直线合并）

场景：main 分支在创建 feature 后没有新的提交。

合并前：

```

main:    A --- B
          \
feature:    C --- D

```

合并（git merge feature）后：

```

main:    A --- B --- C --- D
# Git 只是把 main 指针往前移动了，没有创建新的合并提交

```

```

1 # Fast-Forward 合并
2 git switch main
3 git merge feature          # main 直接移到 feature 的位置

```

4.4.2 Three-Way 合并（分叉合并）

场景：main 和 feature 都有新的提交。

合并前：

```

main:    A --- B --- C
          \
feature:    D --- E

```

合并后：

```

main:    A --- B --- C --- F (merge commit)
          \          /

```

```
feature:      D --- E
# Git 会创建新的合并提交 F，它有两个父提交
```

```
1 # 如果想禁用 Fast-Forward (显式创建合并提交)
2 git merge --no-ff feature      # 即使可以 Fast-Forward 也创建合并节点
3 # 在团队协作中 --no-ff 更常用，因为保留了分支历史
```

4.5 合并冲突

当两个分支修改了同一个文件的同一行时，Git 无法自动判断哪个是正确版本，需要你手动解决。

4.5.1 制造冲突的示例

```
1. 在 main 分支上修改 README.md 的第一行:
   echo "Main Branch Line" > README.md
   git add . && git commit -m "main 修改"

2. 在 feature 分支上修改同一个文件的同一行:
   git switch -c feature
   echo "Feature Branch Line" > README.md
   git add . && git commit -m "feature 修改"

3. 切换回 main 合并 feature:
   git switch main
   git merge feature
   # 输出: CONFLICT (content): Merge conflict in README.md
```

4.5.2 冲突标记

```
1 # 冲突文件里的内容:
2 <<<<<<< HEAD
3 Main Branch Line      # 当前分支 (main) 的内容
4 =====
5 Feature Branch Line   # 要合并分支 (feature) 的内容
6 >>>>>>> feature
7
8 # 你需要手动编辑文件，决定保留什么
```

4.5.3 解决冲突的完整步骤

```
1 # 1. 查看哪些文件冲突了
2 git status
3 # 输出: both modified:  README.md
4
5 # 2. 编辑冲突文件, 删除冲突标记, 保留你想要的内容
6 # 比如你决定两个都要:
7 # Main Branch Line
8 # Feature Branch Line
9
10 # 3. 标记为已解决
11 git add README.md
12
13 # 4. 完成合并
14 git commit
15 # Git 会自动生成合并提交消息, 你也可以修改
16
17 # 5. 如果发现冲突太难解决, 想回到合并前
18 git merge --abort
```

4.6 Rebase (变基)

Rebase 的原理是: 把当前分支的提交移植到另一个分支的顶端。

4.6.1 Rebase vs Merge

```
# Merge 的方式 (保留分叉历史):
# main:  A --- B --- C --- F (merge commit)
#           \           /
# feature:      D --- E

# Rebase 的方式 (线性历史):
# main:  A --- B --- C
#           \
# feature:      D' --- E' (新提交, 哈希变了)
# 然后把 main fast-forward 到 E'
# main:  A --- B --- C --- D' --- E'
```

```

1 # Rebase 操作
2 git switch feature
3 git rebase main          # 把 feature 的提交搬到 main 的最新提交之后
4
5 # 如果 rebase 过程中有冲突:
6 # 1. 解决冲突
7 # 2. git add <file>
8 # 3. git rebase --continue
9 # 或者 git rebase --abort 放弃 rebase

```

4.6.2 交互式 Rebase ——改写历史的瑞士军刀

```

1 # 修改最近 3 次提交
2 git rebase -i HEAD~3
3 # 这会打开一个编辑器，显示:
4 pick 1a2b3c4 第一个提交
5 pick 2b3c4d5 第二个提交
6 pick 3c4d5e6 第三个提交
7
8 # 把 pick 改成对应的命令:

```

可用命令:

- pick (或 p) —保留该提交
- reword (或 r) —只改提交消息，不改内容
- edit (或 e) —修改提交内容
- squash (或 s) —合并到上一个提交 (保留提交消息)
- fixup (或 f) —合并到上一个提交 (丢弃消息)
- drop (或 d) —删除该提交
- exec (或 x) —在此处执行 shell 命令

常见用法: 把一堆零碎的提交合并成一个整洁的提交。

```

# 改前 (杂乱的提交历史):
fix typo
add comment
finish feature

```

```
# rebase -i 改成:
pick finish feature
fixup fix typo          # 合并到上一个
fixup add comment       # 合并到上一个

# 改后（整洁的历史）:
finish feature
```

4.6.3 黄金法则——永远记住！

绝不对已经推送到远程仓库的提交执行 rebase！

因为 rebase 会改变提交的哈希值，如果你 rebase 了已推送的提交，下次 `git push` 会被拒绝（因为历史不一致）。即使强制推送（`git push --force`）成功，队友的仓库也会陷入混乱。

4.7 Cherry-Pick ——精准挑选提交

有时候你不想合并整个分支，只想挑某个分支中的一个提交拿过来。

```
1 # 把 feature 分支上的某个提交（哈希 abc123）应用到当前分支
2 git cherry-pick abc123
3
4 # 一次挑多个
5 git cherry-pick abc123 def456
6
7 # 如果有冲突，解决后：
8 git cherry-pick --continue
9 # 或者放弃：
10 git cherry-pick --abort
```

使用场景：在 main 分支上发现了 feature 分支中的一个 bug 修复提交，没必要合并整个 feature 分支，用 cherry-pick 把那个修复提交拿过来即可。

5 远程仓库——和世界协作

5.1 远程仓库的概念

远程仓库（remote）是存放在**服务器**上的 Git 仓库。常见的有：

- GitHub（全球最大的代码托管平台）
- GitLab（企业级自托管方案）
- Gitee（国内代码托管平台）

远程仓库让你的代码：

1. 有了**备份**——电脑坏了也不怕
2. 可以**协作**——团队多人共同开发
3. 方便**分享**——开源项目

5.2 克隆已有仓库

```
1 # 克隆到当前目录（会创建同名文件夹）
2 git clone https://github.com/username/repo.git
3
4 # 克隆到指定目录名
5 git clone https://github.com/username/repo.git my-folder
6
7 # 克隆指定分支
8 git clone -b feature https://github.com/username/repo.git
```

5.3 管理远程仓库

```
1 # 查看已经配置的远程仓库
2 git remote -v
3 # 输出：
4 # origin https://github.com/username/repo.git (fetch)
5 # origin https://github.com/username/repo.git (push)
6
7 # 添加新的远程仓库
```

```
8 git remote add origin https://github.com/username/repo.git
9
10 # 删除远程仓库
11 git remote remove origin
12
13 # 重命名远程仓库
14 git remote rename origin upstream
15
16 # 查看远程仓库的详细信息
17 git remote show origin
```

origin 是远程仓库的默认名字，就像 main 是默认分支名一样。你可以改，但约定俗成就用 origin。

5.4 推送——git push

```
1 # 首次推送（必须用 -u 建立关联）
2 git push -u origin main
3 # -u 是 --set-upstream 的缩写
4 # 之后就可以直接用 git push 了
5
6 # 推送其他分支
7 git push origin feature-login
8
9 # 推送所有分支
10 git push --all
11
12 # 删除远程分支
13 git push origin --delete feature-login
14
15 # 强制推送（非常危险！会覆盖远程历史）
16 git push --force origin main
17 # 只在确定没有其他人使用这个分支时才用
```

5.5 拉取——git pull 与 git fetch

这是初学者最容易混淆的地方。

- git fetch —只下载，不合并

- 从远程下载最新的提交到你的本地仓库
 - 但不会修改你的工作区
 - 你需要手动用 `git merge origin/main` 来合并
 - 安全，可以随时查看远程有什么变化后再决定
- `git pull` — 下载并自动合并
 - `= git fetch + git merge`
 - 一步到位
 - 但可能会产生多余的合并提交
 - `git pull --rebase` — 推荐！
 - `= git fetch + git rebase`
 - 拉取远程更新后，把你本地的提交放在最上面
 - 保持历史线性整洁

```
1 # 查看远程有更新但不去合并
2 git fetch origin
3
4 # 查看本地和远程的区别
5 git log main..origin/main --oneline
6
7 # 确认无误后合并
8 git merge origin/main
9
10 # 或者一步到位（推荐用 rebase）
11 git pull --rebase origin main
12
13 # 如果 pull 有冲突，按常规冲突解决流程走
```

5.6 标准协作流程

```
1 # 假设你在一个团队项目中
2
3 # 1. 第一天：克隆项目
4 git clone https://github.com/team/project.git
5 cd project
```

```

6
7 # 2. 创建功能分支开发
8 git switch -c feature-user-page
9
10 # 3. 开发过程中，每天上班先同步 main
11 git switch main
12 git pull --rebase          # 获取最新的 main
13 git switch feature-user-page
14 git rebase main           # 你的分支也同步 main 的更新
15
16 # 4. 提交你的工作
17 git add .
18 git commit -m "feat: 添加用户页面布局"
19 git push origin feature-user-page
20
21 # 5. 去 GitHub 上创建 Pull Request (PR)
22 # 等待 code review...
23
24 # 6. PR 合并后，删除本地分支
25 git switch main
26 git pull --rebase
27 git branch -d feature-user-page

```

5.7 SSH 与 HTTPS 的区别

特性	HTTPS	SSH
克隆地址	https://github.com/...	git@github.com:...
认证方式	每次 push 需要输入用户名 密码	密钥对认证
配置难度	简单，无需额外配置	需要生成 SSH 密钥并添加到 GitHub
推荐场景	快速开始、偶尔使用	日常频繁使用

```

1 # 配置 SSH (以 GitHub 为例)
2 # 1. 生成密钥对
3 ssh-keygen -t ed25519 -C "your_email@example.com"
4
5 # 2. 查看公钥

```

```

6 cat ~/.ssh/id_ed25519.pub
7
8 # 3. 复制公钥，添加到 GitHub Settings > SSH and GPG keys
9
10 # 4. 测试连接
11 ssh -T git@github.com
12 # 输出: Hi username! You've successfully authenticated...

```

6 标签管理——标记重要版本

标签用来给某个特定的提交打上有意义的名称，通常用于版本发布。

6.1 轻量标签 vs 附注标签

- 轻量标签 (lightweight)
 - 只是一个指向提交的指针，像一个不会移动的分支
 - 创建: `git tag v1.0.0`
 - 适合临时标记或私用
- 附注标签 (annotated)
 - 作为完整对象存储在 Git 数据库中
 - 包含: 创建者、邮箱、日期、标签消息
 - 可以用 GPG 签名验证真伪
 - 创建: `git tag -a v1.0.0 -m "发布 1.0.0"`
 - 推荐用于正式发布

```

1 # 创建附注标签 (推荐)
2 git tag -a v1.0.0 -m "正式发布 1.0.0"
3
4 # 为历史提交打标签
5 git tag -a v0.9.0 abc123 -m "发布前的 beta 版本"
6
7 # 查看标签
8 git tag                                # 列出所有标签
9 git tag -l "v1.*"                     # 搜索 v1.x 的标签

```

```

10 git show v1.0.0                # 查看标签详细信息
11
12 # 推送标签
13 git push origin v1.0.0          # 推送一个标签
14 git push origin --tags          # 推送所有未推送的标签
15
16 # 删除标签
17 git tag -d v1.0.0              # 删除本地
18 git push origin --delete v1.0.0 # 删除远程
19
20 # 切换到标签时创建分支（因为标签是只读的）
21 git switch -c release-v1 v1.0.0

```

语义化版本控制 (SemVer):

```

vMAJOR.MINOR.PATCH
|      |      |
|      |      +-- PATCH: 修复 bug, 向后兼容
|      +--- MINOR: 添加功能, 向后兼容
+--- MAJOR: 不兼容的重大变更

```

7 储藏 (Stash) ——临时保存工作现场

7.1 什么时候需要 Stash?

你正在开发一个功能，改了 5 个文件，突然需要切换到另一个分支修一个紧急 bug。但这些改动还没完成，不想提交。怎么办？用 `git stash`。

```

1 # 开发中...突然有紧急任务
2 git stash                # 保存工作区所有修改，工作区恢复干净
3 # 输出: Saved working directory and index state WIP on main: abc123 ...
4
5 # 切换到其他分支修 bug
6 git switch hotfix
7 # ...修完 bug 后...
8
9 # 切回来恢复工作现场
10 git switch main
11 git stash pop            # 恢复并删除栈顶的储藏

```

7.2 Stash 的更多用法

```
1 # 储藏时加说明（方便查找）
2 git stash save "用户列表页 WIP"
3
4 # 查看所有储藏
5 git stash list
6 # 输出：
7 # stash@{0}: On main: 用户列表页 WIP
8 # stash@{1}: On main: 登录功能开发中
9
10 # 恢复但不删除
11 git stash apply                # 恢复最近一个
12 git stash apply stash@{1}      # 恢复指定的
13
14 # 删除指定储藏
15 git stash drop stash@{0}
16
17 # 清空所有
18 git stash clear
19
20 # 储藏未跟踪的文件（默认只会储藏已跟踪的文件）
21 git stash -u                  # -u = --include-untracked
22
23 # 从某个储藏中创建新分支（如果恢复时有冲突，推荐用这个）
24 git stash branch new-branch stash@{0}
```

Stash 是一个栈结构，后保存的在栈顶（stash@{0}）。

8 .gitignore ——告诉 Git 忽略什么

8.1 为什么要忽略文件？

你的项目里有很多文件是不需要被版本控制的：

- 编译产物（.pyc、.o、.exe）
- 依赖目录（node_modules/、vendor/）
- 系统文件（.DS_Store、Thumbs.db）

- 环境配置（.env、*.local）
- IDE 配置（.vscode/、.idea/）
- 日志文件（*.log）

不忽略这些文件会导致：

- 仓库臃肿（node_modules 可能有几百 MB）
- 提交混乱（编译文件每个版本都变化）
- 安全风险（.env 可能包含密码）

8.2 语法规则

```

1 # 注释以 # 开头
2
3 # 1. 忽略特定文件
4 .DS_Store           # 忽略当前目录下的 .DS_Store
5 .env                # 忽略 .env
6
7 # 2. 通配符
8 *.log               # 忽略所有 .log 结尾的文件
9 debug.log           # 但 .gitignore 也可以针对具体文件
10
11 # 3. 忽略目录（注意结尾的 /）
12 node_modules/       # 忽略 node_modules 目录及其所有内容
13 build/              # 忽略 build 目录
14
15 # 4. 取反（不忽略某个特定文件）
16 *.log               # 忽略所有 .log
17 !important.log       # 但不忽略 important.log
18
19 # 5. 只忽略根目录下的（不匹配子目录）
20 /ToDo.txt           # 只忽略根目录下的 ToDo.txt
21                     # 不忽略 src/ToDo.txt
22
23 # 6. 忽略所有目录下某类文件，但不包括子目录中的
24 */build/            # 匹配 a/build/ 但不匹配 a/b/build/
25
26 # 7. 忽略除某文件外的所有文件（白名单模式）

```

```
27 /*
28 !src/
29 !src/**/*.py          # 只跟踪 src 目录下的 .py 文件
```

8.3 常见语言/项目的.gitignore 模板

```
1 # Python 项目
2 __pycache__/
3 *.py[cod]
4 *.so
5 *.egg-info/
6 dist/
7 build/
8 .venv/
9 env/
10
11 # Node.js 项目
12 node_modules/
13 npm-debug.log*
14 dist/
15 build/
16 .next/
17
18 # Java 项目
19 *.class
20 *.jar
21 *.war
22 target/
23 !.mvn/wrapper/maven-wrapper.jar
24
25 # LaTeX 项目（你的笔记项目用这个）
26 *.aux
27 *.log
28 *.out
29 *.toc
30 *.fdb_latexmk
31 *.fls
32 *.synctex.gz
```

重要提醒：如果文件已经被 Git 跟踪了，再加入.gitignore 是不生效的！需要用 `git rm --cached` 先解除跟踪。

9 实用技巧与最佳实践

9.1 别名（Alias）——少打字

```
1 git config --global alias.st status
2 git config --global alias.co checkout
3 git config --global alias.br branch
4 git config --global alias.ci commit
5 git config --global alias.unstage "reset HEAD"
6 git config --global alias.last "log -1 HEAD"
7
8 # 设置完后, git st 就等于 git status
9 git st
10
11 # 高级别名: 显示漂亮的日志
12 git config --global alias.tree "log --graph --oneline --all"
13 # 用法: git tree
14
15 # 更美观的版本
16 git config --global alias.lg "log --graph --pretty=format:'%Cred%h%Creset
    -%C(yellow)%d%Creset %s %Cgreen(%cr) %C(bold blue)<%an>%Creset' --
    abbrev-commit"
```

9.2 查看日志的美化方式

```
1 # 一行一个提交, 带图形
2 git log --graph --oneline --all
3
4 # 自定义格式 (可以直接设为别名)
5 git log --graph --pretty=format:'%Cred%h%Creset -%C(yellow)%d%Creset %s %
    Cgreen(%cr) %C(bold blue)<%an>%Creset' --abbrev-commit
6 # 效果: 红色哈希 - 黄色分支名 提交消息 绿色时间 <蓝色作者>
```

9.3 常见场景速查

```
1 # 1. 想在 main 上修改, 但已经开发了一半
2 git stash                                # 保存当前修改
3 git switch main                          # 切回 main
4 # ...做完紧急修改并提交后...
5 git switch feature                       # 切回功能分支
6 git stash pop                            # 恢复工作现场
7
8 # 2. 提交到了 main, 但应该提交到 feature 分支
9 git branch feature                       # 在当前提交创建 feature 分支
10 git reset --hard HEAD~1                 # main 退回一步
11 git switch feature                       # 切到 feature (提交还在)
12
13 # 3. 想修改倒数第二次的提交消息
14 git rebase -i HEAD~3
15 # 把第二行的 pick 改成 r (reword), 保存退出后修改消息
16
17 # 4. 把多个零碎提交合并成一个
18 git rebase -i HEAD~5
19 # 除了第一个 pick, 后面的都改成 s (squash) 或 f (fixup)
20
21 # 5. 想看看某个文件的历史修改
22 git log -p -- README.md                 # 看 README.md 的完整修改历史
23 git blame README.md                     # 每行代码是谁改的
24
25 # 6. 批量修改最近的提交信息
26 git rebase -i HEAD~5
27 # 把要改的 pick 改成 r (reword)
28
29 # 7. 找回误删的分支
30 git reflog                              # 查看所有 HEAD 移动的历史
31 # 输出: abc123 HEAD@{0}: Branch: deleted
32 git switch -c recovered-branch abc123   # 在哈希处创建新分支
```

9.4 救命的 reflog

`git reflog` 是 Git 的操作日志, 记录了 HEAD 的所有移动记录。即使你 `reset`、`rebase` 搞乱了, 也能通过 `reflog` 找回。

```

1 # 查看最近的操作记录
2 git reflog
3 # 输出:
4 # abc123 (HEAD -> main) HEAD@{0}: commit: 修复登录 bug
5 # def456 HEAD@{1}: reset: moving to HEAD~1
6 # ghi789 HEAD@{2}: commit: 测试功能
7 # jkl012 HEAD@{3}: commit: 添加用户页面
8 # 注意: reflog 只记录本地操作, 每个仓库的 reflog 都不同
9
10 # 最实用的场景: 找回 reset --hard 丢失的提交
11 git reset --hard HEAD~2      # 完了, 回退太远了!
12 git reflog                  # 查看历史
13 git reset --hard ghi789      # 恢复回去

```

9.5 二分查找——git bisect

当你发现一个 bug, 但不知道是哪个提交引入的, 可以用 bisect 二分查找。

```

1 # 假设你有 100 个提交, 手动排查要查 100 次
2 # 用二分法最多查  $\log_2(100) = 7$  次
3
4 # 开始二分查找
5 git bisect start
6
7 # 标记当前版本有 bug
8 git bisect bad
9
10 # 标记某个历史版本没有 bug (比如 50 个提交前的版本)
11 git bisect good abc123
12
13 # Git 会自动切换到中间的某个提交
14 # 你测试这个版本, 然后标记:
15 git bisect good      # 如果这个版本没有问题
16 git bisect bad       # 如果这个版本有 bug
17
18 # Git 会继续二分切换到下一个要测试的提交
19 # 重复 good/bad...
20
21 # 最终 Git 会告诉你是哪个提交引入的 bug

```

```
22 # abcdef is the first bad commit
23
24 # 结束 bisect
25 git bisect reset
```

自动化 bisect: 如果项目有测试脚本，可以让 Git 自动运行：

```
1 git bisect start
2 git bisect bad
3 git bisect good abc123
4 git bisect run python test.py    # 自动：测试通过 = good, 失败 = bad
```

9.6 子模块 (Submodule)

当一个 Git 仓库需要引用另一个 Git 仓库时（比如项目引用了某个第三方库），可以使用 submodule。

```
1 # 添加子模块
2 git submodule add https://github.com/lib/lib.git libs/lib
3
4 # 克隆包含子模块的项目
5 git clone --recurse-submodules https://github.com/project.git
6
7 # 更新子模块
8 git submodule update --remote
9
10 # 注意：子模块默认不会自动更新，需要手动更新
```

9.7 提交信息规范——Conventional Commits

为什么要规范提交消息？

- 方便生成 changelog
- 更好的搜索和过滤
- 自动化的版本号管理

标准格式：

```
1 <type>(<scope>): <subject>
2
```

```
3 <body>
4
5 <footer>
```

常用类型:

类型	含义	示例
feat	新功能	feat: 添加用户注册功能
fix	修复 bug	fix: 修复密码验证越界问题
docs	文档变更	docs: 更新 API 文档
style	代码格式	style: 格式化代码缩进
refactor	重构	refactor: 拆分用户验证模块
test	测试相关	test: 添加登录测试用例
chore	构建/工具	chore: 更新依赖版本
perf	性能优化	perf: 优化图片加载速度

好例子 vs 坏例子:

```
# 坏例子
fix bug
update file
asdfgh

# 太模糊
# 说了等于没说
# 毫无意义

# 好例子
fix(login): 修复密码为空时程序崩溃
feat(user): 添加用户头像上传功能
docs(readme): 更新安装步骤, 添加 Windows 系统支持
refactor(db): 提取数据库连接为独立模块
chore: 更新 ESLint 配置, 统一代码风格
```

9.8 Git 钩子 (Hooks)

Git 钩子是在特定事件发生时自动触发的脚本, 放在 `.git/hooks/` 目录下。

常见钩子:

- `pre-commit`: 提交前运行, 用于检查代码风格、运行测试
- `commit-msg`: 检查提交消息格式是否符合规范

- pre-push: 推送前运行
- post-merge: 合并后自动执行

```

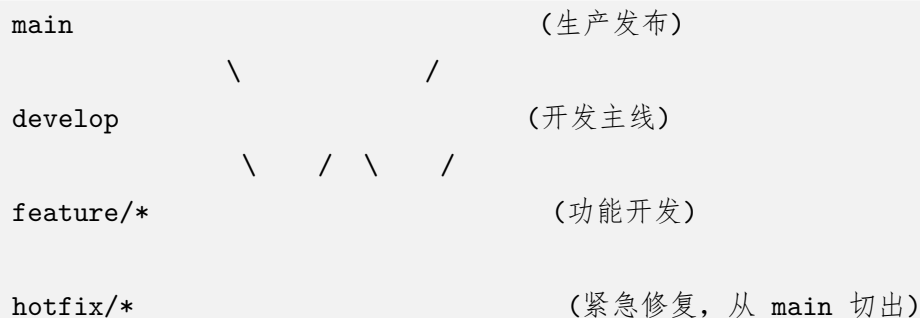
1 # 示例: pre-commit 钩子 (.git/hooks/pre-commit)
2 #!/bin/sh
3 # 检查是否有未解决的冲突标记
4 if grep -r "^<<<<<<<\\|^>>>>>>>" --include="*.py" .; then
5     echo "错误: 存在未解决的合并冲突! "
6     exit 1
7 fi

```

注意: .git/hooks/ 下的钩子不会被提交。团队共享钩子需要借助工具 (如 Husky)。

9.9 工作流模型

9.9.1 Git Flow —— 适合有固定发布周期的项目



- main — 生产分支, 始终处于可部署状态
- develop — 开发主分支, 集成所有功能
- feature/* — 从 develop 创建, 完成后合并回 develop
- release/* — 发布前准备, 只修 bug 不加功能
- hotfix/* — 从 main 创建, 修复后合并回 main 和 develop

优点: 流程清晰, 适合大项目。

缺点: 分支多且复杂, 不适合持续部署。

9.9.2 GitHub Flow —— 适合持续部署

```

main                                     (随时可部署)
      \      /      /
feature      /
      \      /
fix

```

- main 是唯一长期分支，始终可部署
- 从 main 创建特性分支
- 提交修改，频繁推送
- 创建 Pull Request 进行 Code Review
- Review 通过后合并到 main
- 合并后立即部署

优点：极其简单，适合 CI/CD。

缺点：不适合需要多个发布版本同时维护的项目。

9.9.3 GitLab Flow

GitLab 的变体，在 GitHub Flow 基础上增加了环境分支：

```

main
      \      \
staging      \
              \
production

```

环境分支（staging、production）确保代码经过充分测试再上线。

10 内部原理速览——Git 是如何工作的？

10.1 .git 目录全解析

```

.git/
|-- HEAD          # 指向当前分支（如 ref: refs/heads/main）
|-- config        # 本仓库的 Git 配置
|-- description   # 仓库描述（给 GitWeb 用的）

```

```

|-- index                # 暂存区（二进制文件）
|-- ORIG_HEAD            # 某些危险操作前 HEAD 的备份
|-- FETCH_HEAD           # 最近一次 fetch 的来源信息
|
|-- objects/             # 对象存储（Git 的核心！）
|   |-- 1a/              # 哈希前两位做目录名
|   |   `-- b2c3d4e...   # 文件名 = 后 38 位
|   |-- pack/            # 打包文件（压缩优化）
|   `-- info/            # 包文件索引
|
|-- refs/                # 引用
|   |-- heads/           # 本地分支（每个文件存一个哈希）
|   |   |-- main
|   |   `-- feature
|   |-- tags/            # 标签
|   |   `-- v1.0.0
|   `-- remotes/         # 远程分支
|       `-- origin/
|           `-- main
|
`-- hooks/               # Git 钩子脚本示例
    |-- pre-commit.sample
    |-- commit-msg.sample
    `-- ...

```

10.2 底层命令实验

理解 Git 的最佳方式是用底层命令亲手操作：

```

1 # 1. 手动创建一个 blob 对象
2 echo "Hello Git" | git hash-object --stdin -w
3 # 输出：哈希值（同时在 .git/objects/ 下创建了文件）
4
5 # 2. 查看对象类型
6 git cat-file -t <hash>
7 # 输出：blob
8
9 # 3. 查看对象内容
10 git cat-file -p <hash>
11 # 输出：Hello Git

```

```
12
13 # 4. 查看暂存区的内容
14 git ls-files --stage
15
16 # 5. 查看不同引用指向的提交
17 git rev-parse HEAD          # 查看 HEAD 的哈希
18 git rev-parse main          # 查看 main 分支的哈希
19 git rev-parse v1.0.0        # 查看标签的哈希
```

10.3 理解 git add、git commit 的底层操作

当你执行 `git add README.md` 时，Git 实际上做了：

1. 用 `README.md` 的内容创建一个 `blob` 对象
2. 更新 `index`（暂存区），记录 `README.md -> 该 blob`

当你执行 `git commit` 时，Git 实际上做了：

1. 基于 `index` 创建一个 `tree` 对象（记录目录结构）
2. 创建一个 `commit` 对象，指向该 `tree`，并记录父提交等元信息
3. 将 `HEAD` 指向这个新的 `commit`

11 常见问题 FAQ

1. Q: 不小心把密码提交到仓库了怎么办？

- 如果还没有 `push`：用 `git reset` 撤销提交
- 如果已经 `push`：立即修改密码！然后 `git push --force` 覆盖历史（但 GitHub/GitLab 上可能已被爬虫抓取）
- 真正的解决方案：以后用 `.gitignore` + 环境变量

2. Q: 为什么我 `git push` 被拒绝了？

- 最常见原因：远程仓库有新的提交你没拉到
- 解决方案：`git pull --rebase` 后再 `push`

3. Q: `merge` 和 `rebase` 到底选哪个？

- 个人开发/小团队：用 `rebase` 保持历史整洁
- 团队协作/公共分支：用 `merge -no-ff` 保留分支信息

- 规则：对**尚未推送**的提交用 rebase，已推送的用 merge

4. Q: git pull 时冲突了怎么办？

- 和普通冲突一样解决，解决后 `git add + git commit`
- 如果不想解决： `git merge --abort` 回到 pull 前的状态

5. Q: 想放弃当前所有修改怎么办？

- `git restore .` 撤销所有未 add 的修改
- `git reset --hard HEAD` 撤销所有未提交的修改
- `git clean -fd` 删除所有未跟踪的文件和目录
- 组合使用： `git reset --hard HEAD && git clean -fd`
- 以上操作均不可恢复，谨慎使用！

6. Q: .gitignore 为什么不生效？

- 检查文件名是否拼写正确（不是.gitingore）
- 检查是否已经被 Git 跟踪了（用 `git ls-files` 查看）
- 如果已跟踪：先用 `git rm --cached` 解除跟踪

12 总结

Git 学习核心要点：

1. **理解三个区域**：工作区、暂存区、仓库是 Git 的核心抽象。所有操作都是这三个区域之间的数据移动。
2. **分支是指针**：创建和切换分支代价极低，所以大胆创建分支，这是 Git 最大的优势。
3. **小步提交，频繁提交**：每次提交只做一件事，提交消息写清楚 为什么。这让你可以精准地回退到任何状态。
4. **拉取用 rebase,合并用 merge:pull --rebase** 保持本地历史整洁,merge --no-ff 保留分支协作信息。
5. **推送前检查**： `git status` 看状态, `git diff` 看内容，确认无误再推送。

6. **.gitignore 尽早配置**：项目一开始就配好，避免跟踪不必要的文件。
7. **遇到问题先 git status**：这是诊断一切问题的起点。
8. **不要惊慌**：Git 的大部分操作都可以撤销。reflog 是你的最后一道防线，几乎可以找回一切。
9. **多用命令行**：GUI 工具虽然方便，但理解 Git 最好的方式还是命令行。
10. **练习是最好的老师**：开一个新仓库，故意制造冲突、练习 rebase、试试 reset 和 revert。用多了自然熟练。

学习 Git 最好的方式就是多用！祝编码愉快！